



Факултет по Математика и Информатика
ПУ "Паисий Хилендарски"

УДОСТОВЕРЯВАНЕ, СЕСИИ И КОНТРОЛ НА ДОСТЪПА

Витали Атиас

Какво всъщност решаваме?

Всеки уеб софтуер трябва да реши 4 фундаментални
проблема:

- Как разпознаваме потребителя?
- Как запазваме идентичността между заявките?
- Как определяме какво му е позволено?
- Как защитаваме този механизъм от атаки?

Тези четири въпроса определят архитектурата на
сигурността.

Authentication – процес на удостоверяване

Authentication е процесът на проверка на идентичността на потребителя. Това означава, че системата проверява дали потребителят е този, за когото се представя.

Обикновено authentication се извършва чрез потребителско име и парола.

Authentication отговаря на въпроса: „Кой е потребителят?“

Authorization – контрол на гостъна

Authorization отговаря на въпроса: „Какво има право да прави потребителят?“

Например, в университетска система има два типа потребители:

- студент
- администратор

Студентът може да вижда само своите оценки.

Администраторът може да вижда оценките на всички студенти.

HTTP Basic Authentication

Това е най-старият и прост метод, дефиниран в RFC 7617

- Сървърът отговаря с 401 Unauthorized и заглавна част WWW-Authenticate: Basic. Браузърът показва вграден диалогов прозорец за име и парола.
- **"Base64 не е криптиране!"** - Това е критична точка. Base64 е просто кодиране. Всеки, който прихване трафика, може да го декодира мигновено (напр. през терминала или онлайн инструмент).
- Проблемът с Lifecycle: Веднъж въведени, браузърът кешира тези credentials и ги изпраща автоматично до затваряне на сесията. Няма вграден механизъм за „Log out“

HTTP Basic Authentication

Пример:

Authorization: Basic base64(username:password)

Подходящо само за:

- Вътрешни API
- Комбинация с HTTPS

HTTP Digest Authentication

Въведен с RFC 7616, за да реши основния проблем на Basic – изпращането на паролата в чист вид.

- Механизъм (Challenge-Response):
 - а. Сървърът праща nonce (number used once) – уникален произволен низ.
 - б. Клиентът не праща паролата, а генерира MD5 или SHA-256 хеш, който включва паролата, nonce-а, метода на заявката (GET/POST) и URI-то.
- Предимство пред Replay атаки: Тъй като се използва nonce, дори нападателят да прихване хеша, той не може да го използва повторно за следваща заявка (ако сървърът сменя nonce-а често).

HTTP Digest Authentication

Digest използва challenge-response модел.

HTTP/1.1 401 Unauthorized

WWW-Authenticate: Digest realm="example_realm",

nonce="abc123",

qop="auth",

algorithm=MD5

Клиентът връща hash:

hash(username + realm + password + nonce)

Предимство:

- Паролата не се изпраща директно.

Form-based Authentication (стандартният модел)

Form-based authentication (удостоверяване чрез HTML форма) представлява механизъм за автентикация в уеб приложения, при който:

- Разработчикът дефинира собствени login страници (HTML форми) за въвеждане на потребителско име и парола.
- Приложението контролира изцяло визуализацията, обработката на грешки и потребителското изживяване.
- Данните се изпращат към сървъра чрез HTTP POST, а сървърът извършва проверката.

Основен проблем: HTTP е *stateless* протокол

HTTP е *stateless* протокол. Това означава, че всяка заявка към сървъра е независима и сървърът не запомня нищо за предишните заявки.

Решението на този проблем е използването на сесии.

Сесии (Sessions) – механизъм за запазване на състояние

След успешен login, сървърът създава сесия.

Сесията представлява обект, който съдържа информация за потребителя.

Сесията има уникален идентификатор, наречен session ID.

Пример:

SESSIONID = abc123xyz

Сървърът съхранява този session ID и го асоциира с потребителя.

Cookies – как браузърът съхранява session ID

Session ID се изпраща на клиента чрез cookie.

Cookie е малък текстов файл, който браузърът съхранява.

Пример за HTTP отговор:

HTTP/1.1 200 OK

Set-Cookie: SESSIONID=abc123xyz

След това браузърът автоматично изпраща този cookie при всяка следваща заявка:

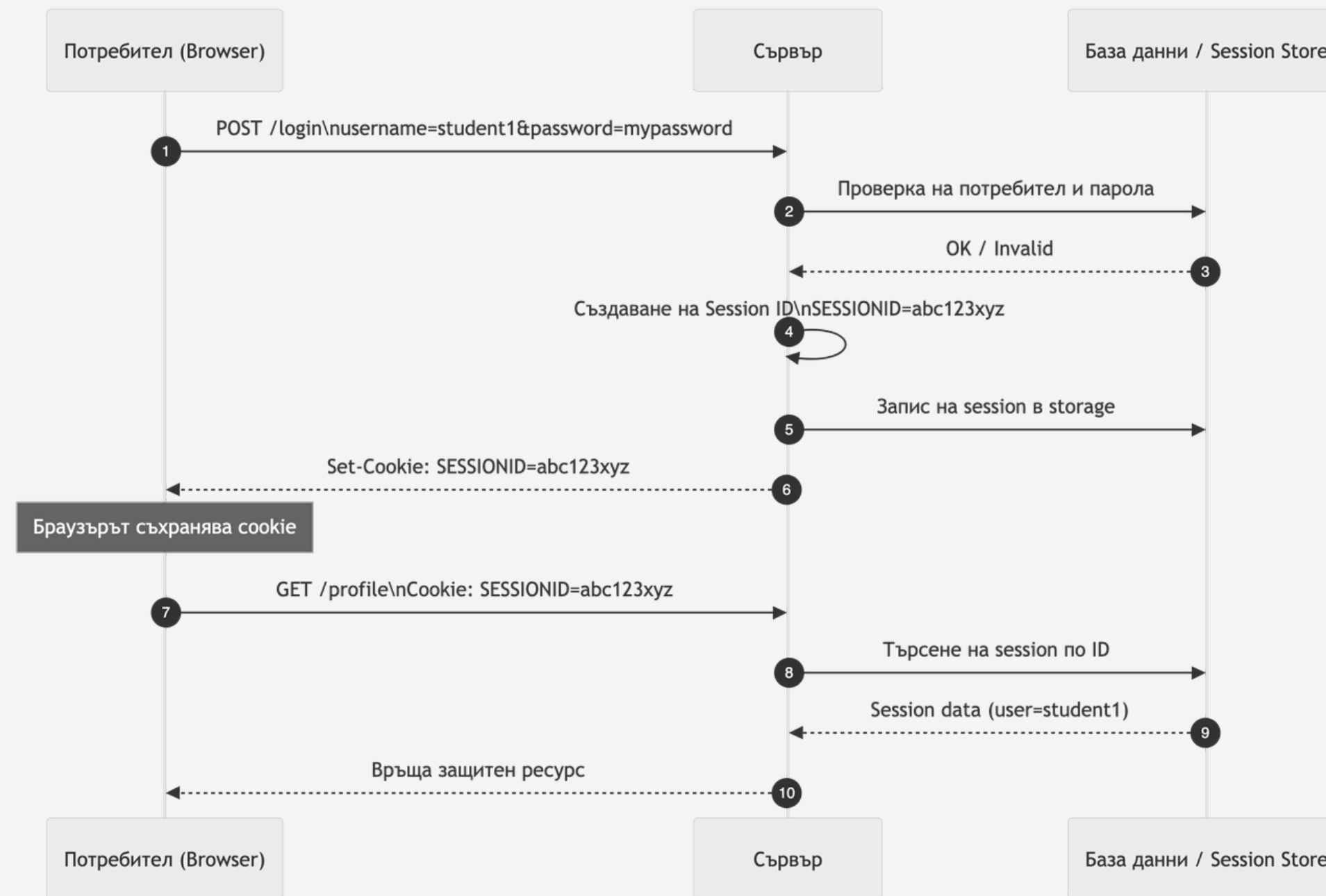
GET /profile HTTP/1.1

Host: example.com

Cookie: SESSIONID=abc123xyz

Сървърът получава SESSIONID и знае кой е потребителят.

Как работи authentication със session



Form-based Authentication (стандартният модел)

Как работи механизмът

1. Потребителят достъпва защитен ресурс.
2. Ако няма валидна сесия → сървърът го пренасочва към login страница.
3. Потребителят попълва HTML форма (username + password).
4. Данните се изпращат към сървъра.
5. При успешно удостоверяване:
 - Създава се сесия (обикновено чрез session cookie).
 - Потребителят получава достъп до ресурса.
6. При неуспех:
 - Пренасочване към страница за грешка или повторен login с message.

Form-based Authentication (стандартният модел)

Предимства на Form-based authentication

- Пълен контрол върху UI/UX.
- Интеграция с:
 - CAPTCHA
 - 2FA
 - Custom validation
 - Localization
- Поддържа сесии.
- Позволява по-добра интеграция със framework-и (Spring Security, Java EE, ASP.NET, Laravel и др.).

Form-based Authentication (стандартният модел)

Недостатъци

- Изисква HTTPS (иначе паролите се предават в чист текст).
- По-сложна имплементация спрямо Basic.
- Изисква правилна защита срещу:
 - CSRF
 - Session fixation
 - XSS

Cookie security flags – защита на cookies

HttpOnly

HttpOnly означава, че cookie не може да бъде достъпен чрез JavaScript.

Без HttpOnly:

document.cookie

може да върне SESSIONID.

Ако има XSS атака, атакуващият може да открадне cookie. HttpOnly предотвратява това.

Пример:

Set-Cookie: SESSIONID=abc123xyz; HttpOnly

Cookie security flags – защита на cookies

Secure flag

Secure означава, че cookie се изпраща само през HTTPS.

Без Secure, cookie може да бъде изпратен през HTTP.

Това позволява interception.

Пример:

Set-Cookie: SESSIONID=abc123xyz; Secure

Cookie security flags – защита на cookies

SameSite flag

SameSite защитава срещу Cross-Site Request Forgery (CSRF or XSRF) атаки.

Set-Cookie: SESSIONID=abc123xyz; SameSite=Strict

Session hijacking – кражба на сесия

Session hijacking означава, че атакуващият получава session ID.

След това той може да се представи за потребителя.

Пример:

Атакуващият получава:

SESSIONID=abc123xyz

След това изпраща заявка:

GET /profile

Cookie: SESSIONID=abc123xyz

Сървърът приема, че това е легитимният потребител.

Session fixation

Session fixation е атака, при която атакуващият задава session ID на жертвата.

След това използва същия session ID.

Пример:

Атакуващият изпраща линк:

<https://example.com/login?SESSIONID=abc123>

Жертвата използва този session ID.

Атакуващият вече знае session ID.

Authentication в SOA / Microservices

В SOA системите имаме:

- Browser → API Gateway
- Service A → Service B
- Background jobs
- Mobile клиенти

Session-based authentication не е достатъчен.

API Keys

Подходящо за:

- Machine-to-machine
- External integrations

Проблем:

- Ако key изтече → пълен достъп
- Няма identity granularity

Пример:

GET /api/orders

X-API-KEY: 3f4a9c8b7

HMAC

Authentication

Предимства:

- Защита срещу tampering
- Защита срещу replay (с timestamp)

Използва се при:

- Финансови API
- Payment providers

Пример:

`signature = HMAC(secret, request_body + timestamp)`

MFA (Multi-Factor Authentication)

2FA представлява механизъм за удостоверяване, при който потребителят трябва да предостави два независими фактора, за да докаже своята самоличност.

Факторите се разделят на три категории:

1. Нещо, което знаеш – парола, PIN
2. Нещо, което притежаваш – телефон, хардуерен токен
3. Нещо, което си – биометрия (пръстов отпечатък, лице)

2FA комбинира два различни типа фактори (например парола + еднократен код).